



UNIVERSITY OF CALOOCAN CITY
COMPUTER ENGINEERING DEPARTMENT



Data Structure and Algorithm

Laboratory Activity No. 14

Tree Structure Analysis

Submitted by:
Regondola Jezreel P.

Instructor:
Engr. Maria Rizette H. Sayo

November 07, 2025

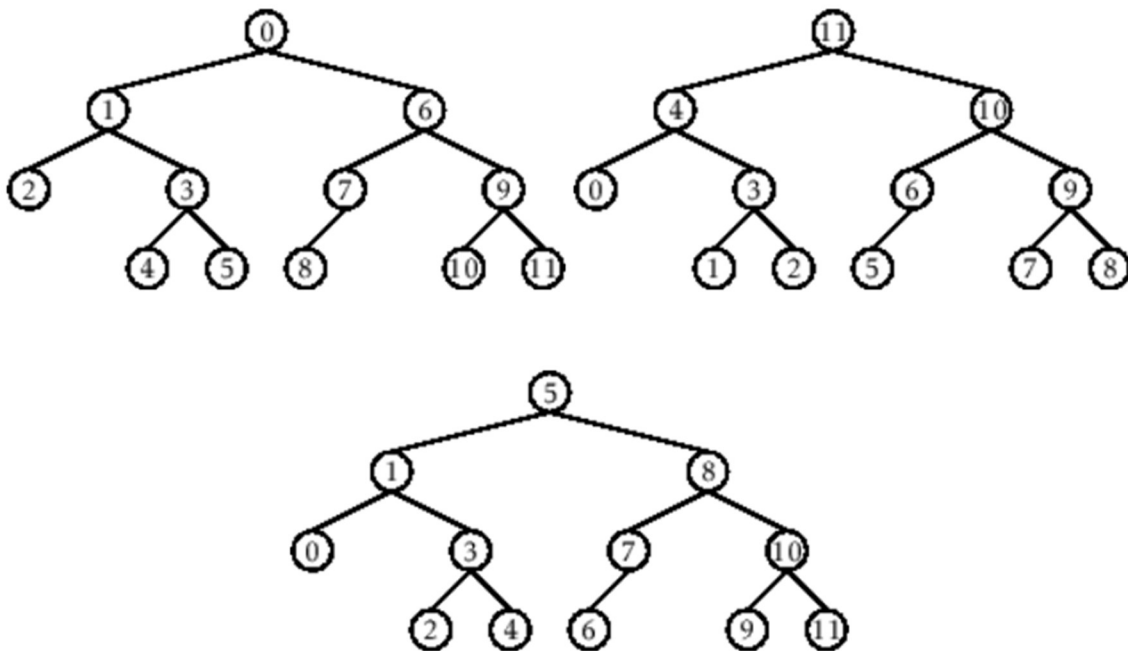
I. Objectives

Introduction

An abstract non-linear data type with a hierarchy-based structure is a tree. It is made up of links connecting nodes (where the data is kept). The root node of a tree data structure is where all other nodes and subtrees are connected to the root.

This laboratory activity aims to implement the principles and techniques in:

- To introduce Tree as Non-linear data structure
- To implement pre-order, in-order, and post-order of a binary tree



- Figure 1. Pre-order, In-order, and Post-order numberings of a binary tree

II. Methods

- Copy and run the Python source codes.
- If there is an algorithm error/s, debug the source codes.
- Save these source codes to your GitHub.
- Show the output

1. Tree Implementation

```
class TreeNode:
```

```

def __init__(self, value):
    self.value = value
    self.children = []

def add_child(self, child_node):
    self.children.append(child_node)

def remove_child(self, child_node):
    self.children = [child for child in self.children if child != child_node]

def traverse(self):
    nodes = [self]
    while nodes:
        current_node = nodes.pop()
        print(current_node.value)
        nodes.extend(current_node.children)

def __str__(self, level=0):
    ret = " " * level + str(self.value) + "\n"
    for child in self.children:
        ret += child.__str__(level + 1)
    return ret

# Create a tree
root = TreeNode("Root")
child1 = TreeNode("Child 1")
child2 = TreeNode("Child 2")
grandchild1 = TreeNode("Grandchild 1")
grandchild2 = TreeNode("Grandchild 2")

root.add_child(child1)
root.add_child(child2)
child1.add_child(grandchild1)
child2.add_child(grandchild2)

print("Tree structure:")
print(root)

print("\nTraversal:")
root.traverse()

```

Questions:

- 1 What is the main difference between a binary tree and a general tree?

- 2 In a Binary Search Tree, where would you find the minimum value? Where would you find the maximum value?
- 3 How does a complete binary tree differ from a full binary tree?
- 4 What tree traversal method would you use to delete a tree properly? Modify the source codes.

III. Results

```
Tree structure:
Root
  Child 1
    Grandchild 1
  Child 2
    Grandchild 2

Traversal:
Root
Child 2
Grandchild 2
Child 1
Grandchild 1
```

Figure 1 Screenshot of the program output

The program demonstrates a general tree where nodes can have multiple children. The structure shows parent-child relationships using indentation. This is different from binary trees which only allow 2 children per node.

1. A binary tree allows each node to have a maximum of 2 children, specifically a left child and a right child. A general tree allows each node to have any number of children without restriction. For example, a binary tree is like having only 2 kids per parent, while a general tree can have many kids per parent.
2. The minimum value is found by starting at the root and continuously moving to the left child until you reach a node with no left child. The maximum value is found by starting at the root and continuously moving to the right child until you reach a node with no right child.

3. A full binary tree requires that every node has either 0 or 2 children, meaning no node can have only a single child. A complete binary tree requires that all levels are completely filled from left to right, though the last level may be incomplete. Full binary tree follows a child count rule while complete binary tree follows a filling rule.
4. Post-order traversal should be used to delete a tree properly because it deletes children first before deleting the parent node.

IV. Conclusion

This lab taught me about different tree types and proper tree deletion. I learned that binary trees limit children to 2, while general trees allow many. Binary Search Trees organize data with small values left and large values right. Post-order traversal is the correct way to delete trees because it removes children before parents, preventing errors.

References

- [1] Co Arthur O.. "University of Caloocan City Computer Engineering Department Honor Code," UCC-CpE Departmental Policies, 2020.